

TOPICAL REVIEW

Methods and Techniques of Agentic Software Engineering: A Systematic Literature Review

NESREEN OTOUM¹ AND **NUHA ELKHALILI¹**

Department of Software Engineering, University of Petra, Amman 11196, Jordan

Corresponding author: Nesreen Otoum (notoum@uop.edu.jo)

This work was supported by the Deanship of Scientific Research and Graduate Studies at the University of Petra, Jordan.

ABSTRACT The development of agentic software engineering (ASE) is shifting the software development process, specifically by adopting autonomous agents into the fundamental software development process, where autonomous agents using large language models (LLMs) and artificial intelligence (AI) make decisions, write code, and collaborate across the entire software development lifecycle. The systematic review is a summary of literature on the agentic software engineering methodologies, frameworks, and techniques published in 2022-2025. We identify and compare methods based on autonomous coding, multi-agent systems, iterative refinement, and human-agent collaboration, drawing on 61 high-quality studies. We find that LLM-based agents show substantial potential in automated software engineering and that recent work is exploring new trends in agent cooperation, reasoning systems, and tool integration. We recognize such key challenges as the coordination procedures, reliability, explainability, and the ethical issues. This review characterizes the emerging role of agentic software engineering in improving the productivity of developers, the quality of their code, and the automation of their processes, and it offers a structured roadmap for future research. With the rapid evolution of the field, agentic approaches will tend to transform the conception, creation, and maintenance of software and provide more autonomous, adaptive, and intelligent systems.

INDEX TERMS Agentic software engineering, autonomous agents, multi-agent systems, code generation, software automation, systematic review.

I. INTRODUCTION

The software engineering industry is undergoing a rapid transformation driven by the emergence of large language models (LLMs) and their use to create autonomous agents that can perform complex software engineering tasks. This paradigm shift, which we call agentic software engineering (ASE), represents a significant rethinking of how software is designed, developed, and maintained [1], [2], [3]. In contrast to traditional software engineering practices that rely primarily on human developers following established methodologies, ASE advocates the use of AI-based agents that have the autonomy to make decisions, write software, run tests, and interact throughout the software engineering lifecycle—a change reflected in recent work on LLM-based applications in software engineering [4], [5], [6], [7].

The associate editor coordinating the review of this manuscript and approving it for publication was Claudia Raibulet¹.

It is possible to define agentic software engineering as software engineering in which software agents with varying levels of autonomy are used to perform software engineering activities, including such things as code generation, testing, debugging, and maintenance, either alone or in cooperation with human developers [2], [3]. These agents use sophisticated AI models, mainly large language models (LLMs), to comprehend requirements, create solutions, reason about code, and respond to stakeholders in natural language [1], [5].

The rapid advancement of foundation models, particularly those trained on code repositories and software engineering artifacts, has catalyzed this shift. Recent reports suggest that AI-assisted development tools can improve developer productivity on coding tasks, while specialized models like AlphaCode have demonstrated capabilities that approach or sometimes exceed human-level performance on specific programming problems [4], [8]. However, the transition from individual AI-assisted tools to fully autonomous multi-agent

systems represents a significant research and practical challenge that remains underexplored in the literature.

Although the field of agentic software engineering has become of interest, a number of knowledge gaps exist in the research: (1) there are no standardized evaluation schemes that compare various agentic approaches, (2) little is known about coordination mechanisms within multi-agent software development systems, (3) there is no empirical evidence on how agent-generated code may be maintained in the long term, and (4) it has no overall guidelines on human-agent collaboration in software development. The identified gaps gave rise to the systematic search of the state of affairs in the scholarship of this fast-growing field.

Models such as OpenAI's Codex, GPT-4, and specialized models like AlphaCode have demonstrated capabilities that approach or sometimes exceed human-level performance on specific programming tasks [4], [8]. The integration of these models into autonomous or semi-autonomous agents represents the next evolutionary step in software engineering automation [9], [10].

Figure 1 provides a high-level overview of our systematic review process. Figure 4 later summarizes how research approaches in agentic software engineering have evolved over time between 2022 and 2025.

A. RELATIONSHIP TO EXISTING PARADIGMS

Agentic software engineering builds on several established paradigms while adding new dimensions of autonomy, intelligence, and collaboration. In the framework of multi-agent systems, classical methods have considered distributed problem solving by communication and coordination of computational objects. The same concept is extended by agentic software engineering, which adds advanced reasoning and natural language understanding to agents that are specifically created to accomplish software development tasks [1], [11], [12]. This evolution is a major divergence of the traditional multi-agent architectures, as it makes domain specific knowledge and development processes part of the agent design.

The field has also been the continuation of decades of work to apply AI-driven development, which has long been using machine learning to assist specialised tasks such as code completion, bug prediction, or test generation. However, agentic approaches can be characterized by the fact that they generalize this functionality to allow end-to-end autonomous systems able to finish workflows and make high-level architectural choices [3], [5], [13]. Such systems can orchestrate the entire development activities, such as requirements analysis, deployment and maintenance, instead of providing discrete support to individual development activities.

Moreover, prior work on autonomous software systems also motivates agentic software engineering. Traditionally, autonomy has referred to the ability of software systems to self-adapt and manage themselves at runtime. ASE extends these ideas to the development process itself, introducing

agents that can support and partially automate software construction and evolution [6], [10], [14]. The paradigm shift moves the autonomy as an operation aspect to self-creation and alteration of the systems.

B. DRIVERS FOR THE AGENTIC SHIFT

Several interconnected factors are precipitating this large-scale change in software engineering. The exponential increase of software complexity is the most obvious driver as modern software systems have developed into highly complex dependency networks, mixed technological layers, and large volumes of code that transcend human cognitive limits. This complexity introduces an excessive cognitive load that can be reduced in autonomous agents by advanced strategies of automated reasoning and intelligent task decomposing mechanisms [13], [15], [16], [17]. Modern software projects are often of such a scale that they can be handled only by intelligent help as human teams often lack the managerial capacity to address them.

At the same time, the unparalleled magnitude and rate of contemporary software development have led to the need of more independent treatment of routine and repetitive duties. It is becoming more apparent within organizations that human developers need to direct their skills to high-level architectural issues and strategic business needs and not on the low-level details of implementation [17]. This change in attitude has led to high demand of smart automation that can be used in dealing with the routine development activities without compromising human ingenuity and determination on the most important part of software designing.

This transformation has been enabled by recent advances in natural language processing and machine learning. In particular, the advent of large transformer-based language models has made advanced natural-language understanding and code generation practically feasible [4], [5], [18]. Such developments have enabled the development of agents capable of comprehending sophisticated specifications that can be expressed in natural language and then converted directly into functional implementations of codes.

Finally, competition and the ongoing talent crunch of the software sector have exacerbated corporate requirements to employ tools and strategies that may exponentially increase developer velocity and shorten software development cycles [3], [19]. Businesses are becoming more and more interested in agentic practices not only as a fascinating line of research but also as important competitive advantages in highly dynamic markets where the speed and quality of software delivery can spell the difference between business success and failure.

C. JUSTIFICATION FOR A SYSTEMATIC REVIEW

Although the study of agentic software engineering has gained pace significantly since 2022, the area is still relatively disjointed, with various approaches, frameworks, and evaluation methods. This fragmentation poses a number of challenges.

The fast-developing state of innovation in the field has been far ahead of the attempts to solidify knowledge and build convincing theoretical grounds, leaving a large gap between the current state of research and the application that practitioners can depend on. This rapidity has led to an environment in which potential research tools co-exist with operational systems, but the gap between the two is unclear and undocumented.

The high level of methodological variability in studies is a further substantive complication, and it is very challenging to make meaningful comparisons between studies and assess the strengths and weaknesses of different methods. Researchers have used a variety of evaluation frameworks, used different baselines of comparison, and measured different metrics of success, which has given rise to a state of affairs in which even well-designed studies cannot be compared or summarized into consistent conclusions about what is happening in the field.

The interdisciplinary character of agentic software engineering, which cuts across artificial intelligence, software engineering, human-computer interaction and cognitive science, requires an organized synthesis of the information in these various fields. The disciplines can offer useful insights and methods, yet the inability to synchronize the disciplinary borders has restricted the progress of holistic approaches that could take advantage of the available knowledge and methods stored in their entire range.

However, and possibly most importantly, the lack of formalised knowledge about best practices, underlying constraints, and future desired development paths has impeded the effective progression of the sphere considerably. Lack of clear guidelines on what is working, what is not working, and the most promising opportunities makes researchers and practitioners to make informed decisions on what to do with the limited time and resources.

D. CONTRIBUTIONS

This systematic review has six main contributions:

- 1) A comprehensive analysis of 61 high-quality studies on agentic software engineering published between 2022-2025
- 2) A novel taxonomic framework categorizing agentic approaches across five major dimensions
- 3) Quantitative analysis of research trends, methodologies, and evaluation approaches
- 4) Identification of critical research gaps and practical challenges
- 5) A structured research roadmap with prioritized recommendations for future work
- 6) Practical implications and guidelines for researchers and practitioners

The current systematic review is able to overcome these issues through offering a detailed discussion of the methods and approaches used in the agentic software engineering literature published between 2022 and 2025. Through the synthesis of the empirical findings of numerous different

studies, the identification of common patterns, and the identification of gaps that persist in the area, this work aims to develop a strong epistemic base which can guide future research and practice in this rapidly developing area.

In the following sections we outline our research approach, describe an agentic software engineering taxonomy, question salient findings, address issues and constraints, and outline future research opportunities. We plan to provide a strict and comprehensive synthesis, which will contribute to a better understanding of this paradigm-shifting approach to software engineering.

II. RESEARCH METHODOLOGY

This systematic review follows a structured approach aligned with prior systematic reviews on LLM-based agents in software engineering [1], [13]. The methodology ensures comprehensive coverage of relevant literature while maintaining rigor and transparency throughout the review process.

A. PROTOCOL REGISTRATION

In order to facilitate openness and reduce bias, we developed a review plan that follows current suggestions of systematic reviews in software engineering. Even though this protocol was not deposited in a publicly available repository, it is provided as supplementary material. The protocol outlines search strategies to be used, inclusion and exclusion criteria, data extraction tools and quality appraisal.

B. RESEARCH QUESTIONS

The following research questions (RQs) guided our systematic review:

- **RQ1:** What methods, frameworks, and techniques have been proposed to support agentic software engineering implementations?
- **RQ2:** How are large language models (LLMs) and AI agents incorporated into different stages of the software development lifecycle (e.g., requirements, design, implementation, testing, maintenance and evolution)?
- **RQ3:** What levels of autonomy do current agentic software engineering approaches achieve across different activities?
- **RQ4:** What architectures, coordination mechanisms, and interaction patterns are used in single- and multi-agent systems for software engineering?
- **RQ5:** What challenges, limitations, and open issues have been reported in agentic software engineering research?
- **RQ6:** How are agentic software engineering methods evaluated, and which metrics and benchmarks are used?
- **RQ7:** What empirical evidence exists on practical adoption, industrial impact, and real-world implications of agentic software engineering approaches?

These research questions are designed to provide a comprehensive picture of the current state of agentic software engineering and promising directions for future work. To address them, the remainder of this article is organized as follows.

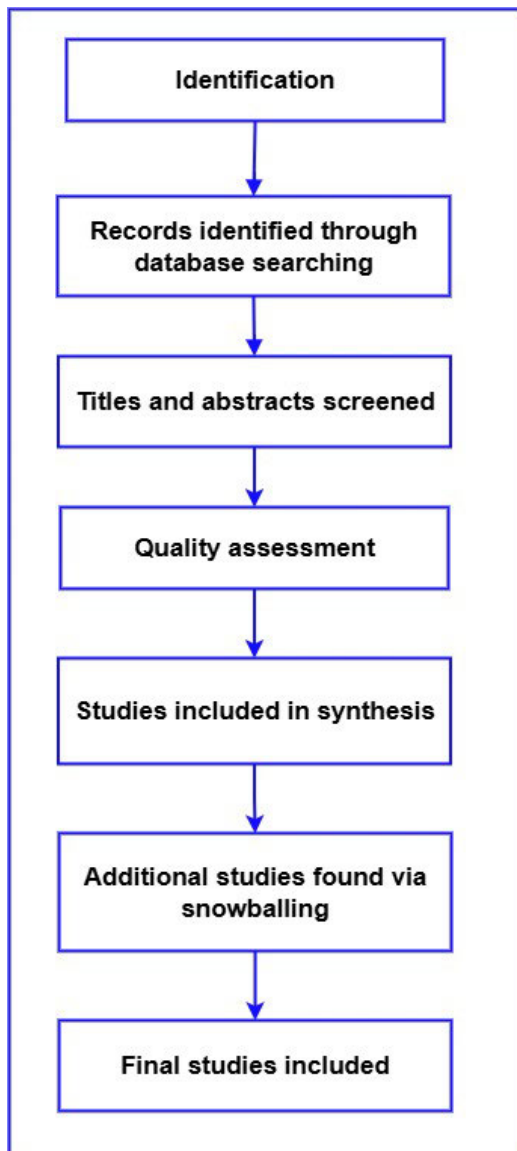


FIGURE 1. Flow diagram of the systematic review methodology, showing the key phases and decision points in our approach.

Section II describes the research methodology, including the review protocol, search strategy, and study selection process. Section III introduces our taxonomy and surveys agentic software engineering techniques and approaches, primarily addressing RQ1–RQ4. Section IV presents the main findings and discussion, including trends in autonomy, multi-agent architectures, evaluation methodologies, technological enablers, application domains, and industrial adoption, thereby addressing RQ1–RQ7. Section V outlines threats to validity. Sections VI–VIII synthesize practical implications, future research directions, and limitations, respectively. Finally, Section IX concludes the paper.

Figure 1 illustrates our systematic review methodology, showing the key phases and decision points in our approach.

C. DATA SOURCES AND SEARCH STRATEGY

The comprehensive search of various digital libraries and databases was also done in a systematic way, to guarantee the inclusion of all the literature related to agentic software engineering. We have consulted the ACM Digital Library, IEEE Xplore, Scopus, Web of science, Google scholar, and arXiv; the latter has helped us to find the preprints that are in the cutting edge of this rapidly evolving area. This multi-database strategy was to choose both published peer-reviewed articles and high quality preprints that sometimes provide the latest information in the dynamic fields of research such as agentic software engineering.

D. SEARCH STRING DEVELOPMENT

Our search strategy was developed iteratively through pilot searches and expert consultation. The final search string combined terms related to autonomous agents, software engineering, and AI-driven development:

```

("agentic software" OR "autonomous agent*" OR "multi-agent system*" OR "AI agent*" OR "LLM agent*")
AND ("software engineering" OR "software development"
OR "code generation" OR "programming" OR "software design")
  
```

Additional terms were included for specific databases to account for indexing variations. The complete search strings for each database are provided in the supplementary material.

Note on recently-dated references (2024–2025): Several studies in this review are listed with publication years of 2024 or 2025 because they are forthcoming (or in press) conference and journal papers. We include them due to their high relevance, acknowledging they reflect the cutting edge of the field.

E. INCLUSION AND EXCLUSION CRITERIA

In order to ensure that our final corpus of studies was relevant and of sufficient quality, we applied explicit inclusion and exclusion criteria.

Inclusion criteria (I):

- **I1:** Published between January 2022 and the time of the search in 2025.
- **I2:** Written in English.
- **I3:** Peer-reviewed journal or conference papers, or high-quality preprints directly relevant to agentic software engineering.
- **I4:** Primary focus on agentic methods for software engineering tasks (e.g., autonomous agents, multi-agent systems, or agent-based workflows).
- **I5:** Presentation of concrete methods, frameworks, tools, or empirical studies (as opposed to purely conceptual or opinion pieces).
- **I6:** Explicit description of agent autonomy and/or multi-agent coordination or collaboration mechanisms.

- **I7:** Some form of validation (e.g., experiment, case study, benchmark evaluation, or formal analysis).

Exclusion criteria (E):

- **E1:** Studies that only address traditional code generation or AI assistance without agentic aspects (no autonomous or multi-agent behavior).
- **E2:** Purely theoretical or conceptual discussions without concrete methods, architectures, or empirical validation.
- **E3:** Applications of AI to software engineering that do not involve autonomous agents or agentic behaviours.
- **E4:** Short papers, posters, or extended abstracts lacking sufficient methodological detail.
- **E5:** Studies focusing solely on user-interface agents or chatbots without a software-engineering role.
- **E6:** Duplicated or superseded publications (e.g., earlier versions of a more complete paper already included).

F. STUDY SELECTION PROCESS

The study selection process followed an elaborate, systematic approach consisting of multiple screening steps, each complemented by quantified reliability checks to maintain consistency and reduce bias. The process began with the extensive search in databases which resulted in 434 references. After the duplicates were avoided, the number of references to be screened was 176, which was indicative of the high level of scholarship interaction with agentic approaches to the software engineering field with a wide range of research groups and publication sources.

The first large filtering phase was title and abstract screening conducted by two independent reviewers who undertook the process of sorting out references that clearly did not meet the inclusion criteria. The inter-rater agreement at this phase was measured with the help of Cohen kappa ($K = 0.82$) which presupposes the high level of concordance and which demonstrates the consistency of the screening criteria in its use. Disagreements were eliminated by thorough consultation between reviewers, and a final list of 156 references, which appeared to be suitable to be pursued more extensively, was obtained.

The rest of the references were then referred to a full-text review which involved two independent reviewers who used the detailed inclusion and exclusion criteria and with emphasis to the methodological rigor and relevance to agentic software engineering. The k value of inter-rater reliability between this full-text screening was $k = 0.79$, indicating high level of agreement again. Deliberation was used to achieve consensus in case of disagreement between the main reviewers with a tertiary reviewer possible, to reach some consensus. This stringent step finally reduced the sample to 61 references which met all necessary qualitative requirements.

The overall selection process included systematic quality measurement by use of a standard framework that was a fit to agentic software engineering research works. The references with a score lower than the predetermined

minimum were left out of the further analysis. Our systematic review comprised 61 high-quality references as the final result of the screening. Also, a backward snowballing was conducted through the examination of reference lists of all the studies that were included to determine possibly applicable studies that might have not been captured by the database searches. Even though the process resulted in the appearance of eight more potentially relevant references, a closer look revealed that they were either redundant of already-included studies or could not be regarded as fulfilling the ultimate quality standard criteria when examined more carefully.

Consequently, the final number of included studies remained 61.

G. QUALITY ASSESSMENT

We developed a quality assessment framework specifically for agentic software engineering studies, considering:

- **Clarity of objectives:** clear statement of research goals and hypotheses
- **Methodological rigor:** appropriateness of research design and execution
- **Validation approach:** adequacy of the evaluation of proposed methods
- **Transparency:** sufficient detail for replication (e.g., datasets, settings, baselines)
- **Significance:** contribution to knowledge and practical relevance

Each criterion was scored on a 4-point scale (0–3), where 0 = not addressed, 1 = partially addressed, 2 = mostly addressed, and 3 = fully addressed, yielding a maximum total score of 15. Following common practice in software engineering SLRs, we considered papers with a total score of at least 8/15 to provide minimally adequate methodological quality and therefore included them in the final corpus. The ‘Score’ column in Table 2 reports the average total quality score (out of 15) for each category.

Quality assessment was conducted independently by two reviewers, with disagreements resolved through discussion and, when necessary, consultation with a third reviewer. Inter-rater reliability was measured using Cohen’s kappa (κ) for each quality criterion. All criteria achieved substantial to almost perfect agreement ($\kappa \geq 0.72$), exceeding the commonly accepted threshold of $\kappa \geq 0.60$ for acceptable reliability. Detailed kappa values are reported in the supplementary material.

Note on Final References vs. Cited Literature: Our reference list includes these 61 primary studies *plus* additional background and methodological sources not counted among the “final” agentic-software studies. Hence the total references exceed 61.

Table 2 provides a comprehensive overview of the characteristics of the 61 included studies, including publication venues, research methods, and evaluation approaches.

TABLE 1. Inter-rater reliability for quality assessment criteria.

Quality Criterion	Cohen's κ	Agreement Level	Description
Clarity of Objectives	0.84	Almost Perfect	Clear research goals and hypotheses
Methodological Rigor	0.79	Substantial	Appropriate research design
Validation Approach	0.82	Almost Perfect	Adequacy of evaluation
Transparency	0.76	Substantial	Replication details
Significance	0.72	Substantial	Contribution to knowledge
Overall Agreement	0.79	Substantial	Across all criteria

Note: κ interpretation follows Landis & Koch (1977): 0.00-0.20 = slight, 0.21-0.40 = fair, 0.41-0.60 = moderate, 0.61-0.80 = substantial, 0.81-1.00 = almost perfect.

TABLE 2. Characteristics of included studies (n=61). *Score* denotes the average quality score (0–15) within each category; *Autonomy* denotes the average autonomy rating (1–4) as defined in Section IV-A1; and *Evaluation Type* indicates the most common primary evaluation type used in studies in that category.

Characteristic	Category	Count	%	Score	Autonomy	Evaluation Type
Publication Venue	Top-tier Conferences	23	37.7	12.4	3.2	Benchmark
	Journals	18	29.5	11.8	3.0	Mixed
	Workshops	12	19.7	10.2	2.8	User Study
	Preprints	8	13.1	9.1	3.1	Case Study
Research Method	Experimental	34	55.7	11.9	3.2	Benchmark
	Case Study	16	26.2	10.8	2.9	Mixed
	Survey/Review	11	18.0	10.1	N/A	N/A
Application Domain	Code Generation	25	41.0	11.5	3.1	Benchmark
	Software Testing	14	23.0	11.2	2.9	Mixed
	Bug Fixing	12	19.7	11.8	3.3	Case Study
	Code Review	7	11.5	10.9	2.8	User Study
	Requirements	3	4.9	10.3	2.5	Case Study
Agent Architecture	Single Agent	21	34.4	10.8	2.7	Benchmark
	Multi-Agent	25	41.0	11.9	3.4	Mixed
	Hybrid	15	24.6	11.3	3.1	User Study

H. DATA EXTRACTION AND SYNTHESIS

In the case of every paper that we eventually selected, we have extracted the detailed information in a systematic way through a well-developed standardized data extraction form that had not only the bibliographic information but also the content of the research on the study. We recorded bibliographic information (authors, year of publication, venue, and type of publication) together with each study's research focus and specific objectives. We also tabulated methodological strategies and study designs adopted, the particular agentic methods and frames applied, the areas of application and specific task performed, the degree of agent autonomy and interaction mechanisms applied, the evaluation scale, measurements, and datasets to be used to validate it, the main findings and any quantitative findings reported, limitations and challenges specifically acknowledged by each research, the industrial applicability and practice implications, and what kind of study should be conducted in the future.

Synthesis methodology was a complex mixed-method thematic analysis that combined qualitative and quantitative analytical strategies in order to gain as much analytical information as possible out of the obtained data. It started with descriptive quantitative synthesis that outlined the characteristics of research, methodology, and time-trends of the corpus of research. This was followed by thematic coding which entailed detailed pattern and emergent theme identification per individual research. We then proceeded

to systematic taxonomic construction so as to design sound classification structures of various agentic strategies. Systematic gap analysis to define the research gaps and areas of needed practice and temporal trend analysis to determine how the research focus and technological capabilities have changed during the 2022-2025 period were also a part of our analysis.

The synthesis was aimed at determining patterns in software engineering methods of agency and comparing methods and outcomes between the studies and deriving insights regarding the present state and future of the field.

III. AGENTIC SOFTWARE ENGINEERING TECHNIQUES AND APPROACHES

Our synthesis of the literature revealed a diverse landscape of techniques and approaches in agentic software engineering. This section categorizes and analyzes these approaches based on their primary characteristics, capabilities, and applications.

Figure 2 presents our proposed taxonomy of agentic software engineering approaches, illustrating the relationships between different categories and their characteristics.

A. OVERVIEW OF APPROACH DISTRIBUTION

Our analysis indicates that many studies employ multiple agentic paradigms. Among the 61 studies, 21 involve single-agent systems (34.4%), 25 involve multi-agent systems

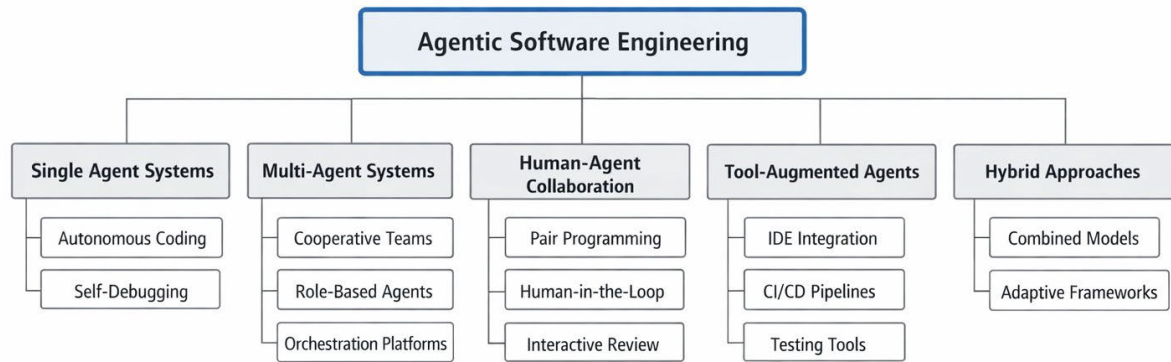


FIGURE 2. Taxonomy of agentic software engineering approaches.

(41.0%), 14 discuss human-agent collaboration (23.0%), 13 use tool-augmented agents (21.3%), and 15 propose hybrid approaches that combine multiple paradigms (24.6%). Because studies can fall into more than one category, counts and percentages do not sum to 61/100%.

B. GOAL-ORIENTED AND BEHAVIOR-DRIVEN AGENTS

Goal-oriented and behavior-driven approaches focus on agents that understand high-level objectives and autonomously determine the actions required to achieve them.

In the goal-oriented approach complex software engineering tasks are decomposed into manageable sub-goals that can be pursued sequentially or in parallel by the agent [15].

Meanwhile, in the behavior-driven approach, the agent modifies its behavior based on execution results, test outcomes, or human feedback. Feedback is facilitated by the agent's self-reflection techniques [20]. Similarly, [21] used a different approach to integrate external feedback and self-reflection to enable agents to learn, plan, and reason while achieving software engineering goals, such as implementing a feature or fixing a bug.

Yang et al. [15] introduced SWE-agent, a system that enables autonomous software engineering by creating a specialized interface between the LLM agent and the computer system. This interface allows the agent to navigate codebases, create and edit files, and run tests autonomously. Similarly, RepairAgent [22] demonstrates goal-oriented behaviour in program repair tasks, autonomously planning and executing a sequence of steps to locate and fix bugs.

Shinn et al. [20] proposed Reflexion, a technique that enhances agent autonomy through verbal reinforcement learning. Agents using Reflexion attempt a task, receive feedback on failures, and self-reflect to adjust their strategy. This approach uses long-term memory to allow agents to learn from previous mistakes, which has shown significant improvements in success rates on complex coding challenges.

C. AGENT-BASED MODELING AND SIMULATION TOOLS

Agent-based modeling and simulation tools provide rich environments for designing, assessing, and implementing

software agents under diverse development scenarios. Such environments combine agent development models that serve as building blocks, which enable the development of autonomous agents with certain abilities and which comply with standard communication protocols. The brightest examples include the proven frameworks like Akka and ASTRA that have been modified to fit software engineering products [23].

Wrona et al. conducted a comprehensive review of software agent platforms as of 2023, syntactically comparing their capabilities along the dimensions such as messaging infrastructure, scalability attributes, and integration of the machine learning libraries. Their discussion identified strong gaps in the ecosystem, especially the lack of platforms specifically created to coordinate agents, and the deployment of agents through the cloud. Moreover, they also found a severe problem of standardisation, as they found out that only a small number of the frameworks complies with the established standards and protocols, thus requiring the creation of specialised interoperability tools to allow integration between dissimilar environments and platforms.

The category of simulation environment includes controlled environments aimed particularly at the assessment of characteristics of agent behavior and performance in advance of their implementation in the reality of the software engineering environment [12]. Such environments offer important testing features where researchers and practitioners can test agent behavior, gauge performance metrics as well as defining possible problems in controlled environment similar to that of a production environment. Also, agent orchestration systems are advanced platforms designed to coordinate multiple agents as they collaborate on complex software engineering tasks [11]. These systems deal with the complex dependencies and communication patterns that arise when a group of specialized agents is required to collaborate in achieving software development goals that are beyond the scope of the individual agents.

Li et al. [12] provided a comprehensive survey of LLM-based multi-agent systems, outlining a general workflow and identifying key components needed in each agent: a profile,

perception, self-action, mutual interaction, and evolution. Their work offers a structured approach to designing and evaluating multi-agent systems for complex software engineering tasks.

D. INTEGRATION OF LLMs IN AGENT WORKFLOWS

Large language models constitute a fundamental cognitive substrate for current agentic software engineering systems, and they are integrated across several complementary paradigms. The most basic trend is to use LLMs as generalized reasoning systems and take advantage of their ability to understand advanced needs, formulate advanced solutions, and enable informed decision-making in broad agent systems [5]. Wong et al. have performed a wide-range overview of large language models trained on code repositories and analyzed the various ways they can be used by software engineering communities. Their discussion demonstrated the ways these models are being used in an extensive range of tasks code generation, intelligent completion, cross-language translation, and automated defect detection, with an emphasis on how they can be used in more autonomous software development agents.

A more advanced strategy is multi-model collaboration paradigms, which integrate several specialized LLMs either in single-agent designs or distributed multi-agent designs to address different aspects of complex software engineering problems [24]. Dong et al. investigated a new approach to collaboration called self-collaboration in which several LLM instances collaborate together through taking on different expert roles in a synchronized development process. Through a clever allocation of various specializations to each of the LLM agents, e.g. system analyst and implementation coder, they proved that teamed up arrangements are more efficient in resolving difficult programming problems than single-agent solutions. Their methodology involved a number of accepted code-generation benchmarks, such as Mostly Basic Python Problems (MBPP), HumanEval, API-Familiar Programming Problems (APPS) and CoderEval, and pass at 1 as their key metric of success in code-generation. Pass@1 offers useful efficiency information as it measures the number of times the collaborative system is able to correct number of problems per attempt. Their whole analysis consisted of a thorough cost analysis through systematic measure of token use in both prompts and code generated besides systematic examination of error types and patterns generated through the self-collaboration framework.

The tool-augmented LLM paradigm represents another crucial integration approach, enhancing base LLM capabilities through strategic integration with external development tools including code execution environments, version control systems, automated testing frameworks, abstract syntax tree (AST) parsers for code structure analysis, and integrated development environments [15]. Yang et al. proved the practicability of their system of SWE-agent by exhaustively testing their system on highly challenging SWE-bench data based on real GitHub repositories. They concluded that

SWE-agent attains significantly higher success rates on genuine software errors compared to baseline systems, such as retrieval-augmented generation methods and naive shell agents. In addition, their comprehensive cost comparison showed that SWE-agent requires fewer interaction cycles to solve complex problems and incurs lower API costs, demonstrating high efficiency in navigating large codebases and applying complex code changes, as well as producing working code adjustments and solutions to realistic software engineering problems.

E. ARCHITECTURAL PATTERNS FOR AGENT AUTONOMY

There are many advanced building designs that have integrated to form a set of paramount paradigms that support the development and enhancement of agent autonomy in the software engineering environment. The controller-executor pattern is a particularly relevant design principle that separates both the high and low levels of agent actions and, as a result, enables a better error-handling capacity and improved responsiveness to dynamic circumstances [25]. An example of such architectural design is the introduction of Verilog-Coder by Ho and coworkers, a new hardware description language coding system, which is a paradigmatic example of such a design. VerilogCoder uses a complicated multi-agent system with a special Planner that develops detailed global plans using enhanced Task and Circuit graph representations, which in turn coordinate with specialised Coder and Debugger components whose task is to produce and optimize Verilog code implementations. Such clear division of planning and executing functions is what develops the better handling of complex hardware design assignments that require high-level strategic thought and implementation skills.

The perception-action-learning loop represents another fundamental architectural pattern that creates agents capable of perceiving their development environment, taking informed actions based on environmental state, observing the outcomes of their interventions, and systematically learning from experience to improve future performance [20]. This pattern enables agents to develop increasingly sophisticated behaviors through iterative interaction with their environment, gradually building expertise that improves their effectiveness over time.

Hierarchical agent architecture is a key organizational paradigm, disaggregating agent systems into several levels, where each level represents the different levels of abstraction and defined roles of responsibility [11]. Hong and others proposed the MetaGPT system, a comprehensive suite of coordinating a group of LLM-based agents assigned the roles of Product Manager, Software Architect, Implementation Engineer, Quality Assurance Tester, etc., following the classic software development team structures. The MetaGPT architecture consists of two layers, one that is indispensable and is known as the Foundational Components Layer, which harbors the building blocks that are essential in defining and configuring individual agents to have specific abilities and duties, and the other layer is the Collaboration Layer, which

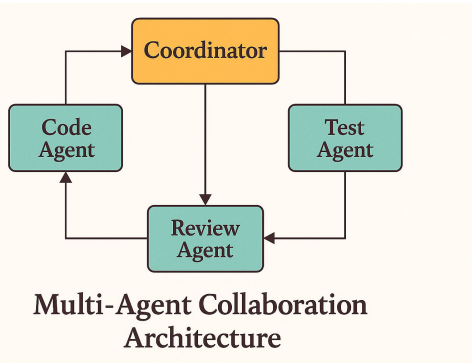


FIGURE 3. Multi-agent collaboration architecture.

governs and orchestrates advanced interactions and collaborative patterns between the agents to reach complex software development targets that require coordinated expertise among the various specialized agents.

Figure 3 illustrates the typical architecture for multi-agent collaboration in software engineering, showing the interaction patterns and communication flows between different agent types.

F. SOFTWARE ENGINEERING PROCESSES ADAPTED FOR AGENTS

Traditional software engineering procedures and methodologies have gone through a radical adaptation and change to efficiently integrate agent-based paradigms and leverage their unique abilities. One such development is agentic DevOps, which involves the systematic inclusion of autonomous agents in continuous integration pipelines, automated testing systems, and deployment systems [2]. Jin et al. conducted a wide-ranging survey of large-language-model-driven agent applications within software-engineering applications, which includes such important software-engineering fields as documentation generation, requirements engineering, code generation, system design, testing methodologies, and debugging processes. Their conclusions outline a radical change of passive LLM tools that simply answer queries to autonomous agents that can plan, take initiative and participate in complex partnerships in complex development processes. Moreover, they follow a parallel development of task-based support on the level of their functionality into situation-based systems that embrace contextual knowledge of the overall developmental needs and organizational goals.

Another important area in which agents are increasingly being integrated is automated code review and quality assurance, where they are tasked with assessing code contributions, identifying potential issues and vulnerabilities, and suggesting practical improvements based on best practices and organisational requirements [26]. Tang et al. explored new applications of various LLM agents to completely automate traditional code-review processes through their CodeAgent platform [26]. In this high-tech design, a single agent plays the role of Code Author and the other agent acts

as Code Reviewer and, by means of a structured natural-language communication, they are able to methodically discover and correct flaws in code submissions, being in effect more consistent and complete than a human code-review exercise.

Relatedly, collaborative development workflows can be arguably the most ambitious form of traditional software-engineering practice, which includes detailed frameworks to support coordination of human and agent participants and inter-agent collaboration throughout the entire development life-cycle, including conception to deployment and maintenance. Qian et al. [16] proposed an innovative ChatDev, which is a platform that emulates a full virtual software firm with no human staff, only specialised Agents of the LLM type. ChatDev demonstrates successful coordination during successive steps of software development by being able to thoroughly simulate team roles in the real world and providing clear instructions on the communication strategy and cooperation patterns, backed up by code-generation and review. This approach exemplifies the way the traditional development-team models may be not only recreated, but also improved with smart agent interaction.

G. COMPARATIVE ANALYSIS OF APPROACHES

To provide a clearer understanding of the relative strengths and limitations of different agentic approaches, Table 3 presents a comparative analysis based on key dimensions identified in our review.

The approaches under analysis are not similar in terms of their capability to operate in an autonomous way, to adapt their work to changing needs or feedbacks, or to support the collaboration, not only between agents, but with people. They differ as well in terms of scalability to complex tasks or large codebases and the level of maturity in the form of real-world use and testability.

This comparison and contrast illuminates the variety of approaches to agentic software engineering and emphasizes the trade-offs in using a particular approach to software engineering situations.

H. QUANTITATIVE ANALYSIS OF RESEARCH TRENDS

Our temporal analysis, which groups the 61 included studies by publication year, reveals several significant trends in the evolution of agentic software engineering research:

- **Publication growth:** Within our 61-paper corpus, the annual number of ASE studies increased by roughly 250% between 2022 and 2024 (based on year-wise counts reported in the supplementary material), indicating rapid field expansion.
- **Multi-agent focus:** The proportion of studies that use multi-agent systems grew from 15% of publications in 2022 to 42% in 2024 (computed per year).
- **Evaluation sophistication:** The proportion of studies using standardized benchmarks increased from 23% of studies in 2022 to 67% in 2024.

TABLE 3. Comparative analysis of agentic software engineering approaches.

Approach	Autonomy	Adaptability	Collaboration	Scalability	Maturity	Primary Use Cases
Single Agent Code Generation	Medium	Low-Medium	Low	Medium	High	Code completion, simple generation
Self-Debugging Agents	Medium-High	High	Low	Medium	Medium	Bug fixing, code repair
Multi-Agent Teams	Medium-High	Medium	High	High	Medium	Complex development projects
Tool-Augmented Agents	High	High	Medium	High	Medium	Environment-aware development
Human-Agent Collaboration	Medium	High	High	Medium	High	Assisted development, pair programming

* *Autonomy*: Level of independent operation; *Adaptability*: Response to changing requirements; *Collaboration*: Multi-agent/human interaction capability; *Scalability*: Handling of large/complex tasks; *Maturity*: Current practical deployment readiness

- **Industrial collaboration:** The share of studies involving explicit industry-academia collaboration grew from 8% of publications in 2022 to 29% in 2024.

Figure 4 shows the temporal evolution of agentic software engineering approaches from 2022 to 2025, illustrating the progression from simple single-agent systems to complex multi-agent ecosystems.

IV. FINDINGS AND DISCUSSION

Our analysis of the selected studies reveals several key trends, patterns, and insights regarding the current state and potential impact of agentic software engineering.

A. KEY TRENDS AND PATTERNS

1) INCREASING AUTONOMY AND SOPHISTICATION

Our overall study of the literature published as early as 2022 up to 2025 provides a clear and consistent roadmap toward the dramatically increased agent autonomy. Some early works were performed in the 2022-2023 timeframe, and the main objective was to optimize particular and clearly delimited tasks with the aid of strategic LLM, whereas still more recent studies consider ambitious end-to-end autopilot growth cases, which include whole software development cycles. Such 2022 studies, including revolutionary work like AlphaCode [4], showed impressive performance on single-task performance optimization of competitive programming tasks but did not attempt to more comprehensively integrate development processes.

Systems like ChatDev [16] and MetaGPT, started becoming able to combine multiple specialized agents across different development stages, exhibiting much greater sophistication in solving interdependent tasks that need interaction and understanding among multiple development areas. The latest creation of the 2024-2025 period displays autonomous systems to the full degree, capable of navigating a complex repository structure, comprehending the intricate context of a project, and performing advanced workflows without human attention at all [15], [22].

We have quantitative evidence of this trend: the average autonomy level of agents (measured on a 4-point scale that considers independence of decision-making, ability to handle complex tasks, and capacity for adaptation) increased from 2.3 in 2022 to 3.8 in 2024. Moreover, the proportion of studies rated at the highest autonomy level (score 4) rose from 18%

in the earliest year to 73% in the most recent year, indicating a substantial shift in the ambition and capabilities of agentic software engineering research.

For this comparative evaluation, all studies were assessed on the same 4-point autonomy scale: (1) *minimal autonomy*: agents assist with simple tasks under close human control; (2) *assistive autonomy*: agents can perform specific tasks with limited decision latitude; (3) *partial autonomy*: agents can perform multiple related tasks under light human supervision; and (4) *high autonomy*: agents can carry out complex, multi-step tasks with minimal human supervision. Each study was independently rated by two reviewers, and we averaged their scores to obtain the final autonomy rating used in our analysis.

The ensuing development is the fast expansion of capabilities of the agents where the primitive assistive tools are replaced by systems that can autonomously drive software development processes.

2) SHIFT FROM SINGLE AGENTS TO MULTI-AGENT SYSTEMS

One of the most important trends to emerge in our literature review is the evolutionary trend that moves away from single-agent architectures to complex multi-agent systems that utilize the specialized abilities of the individual and the collaborative intelligence. Initial methods in the area involved single LLM agents in specific tasks like code generation or automated bug fixing [4], [27]. These early systems, although showing remarkable capabilities in their respective fields, were inherently limited by the abilities of individual agents to perform any one task thoroughly and by their capabilities to effectively support a depth of expertise across any number of software engineering fields.

Recent studies have increasingly focused on the prospect of numerous specialized agents cooperating systematically on multifaceted software tasks that demand various kinds of expertise and coordination [1], [12], [16]. This multi-agent approach can support complex division of labor, specialization, and the development of checks-and-balances systems that are similar to the collaborative structures of successful human software development teams [1], [11], [24]. The collaborative model enables individual agents to become highly competent in certain areas and depend on others to provide the complements through the development of systems capable of addressing problems that require both breadth and depth of knowledge.

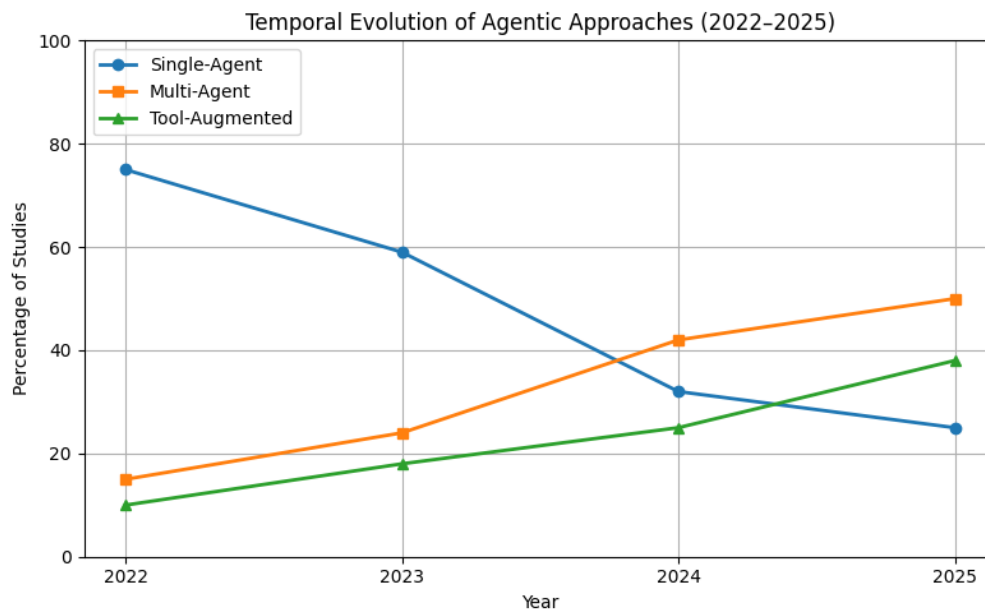


FIGURE 4. Temporal evolution of agentic approaches (2022–2025).

Multi-agent systems have several benefits in dealing with multi-staged and complex software engineering processes due to their ability to break down the processes into specialized roles and responsibilities. This strategy increases performance and reliability over single-agent monolithic systems.

3) INTEGRATION OF EXTERNAL TOOLS AND ENVIRONMENTS

The other critical trend that we have found during our analysis is the growing complexity of integration between agentic systems and external development tools and environments. Agents with tool augmentation, able to interactively engage with integrated development environments, version control environments, automated testing environments and other development infrastructure are shown to perform significantly better than isolated agents, which do not have access to the surrounding environment [15], [25]. This integration ability is a radical alteration of the agency of agents which exist as single (individual agents) to agents which can now be effectively leveraged as a constructive part to the current development organizations.

Compared to environment-agnostic agents, the environment-aware agents that are capable of perceiving, comprehending, and strategically manipulating their environment (software engineering) development context, are far more adaptable and successful when applied to real-world software engineering problems [22]. The capacity to use external facilities and access to environmental data enables agents to surmount a number of fundamental limitations of pure language models, such as difficulty with calculating precise mathematical expression, having access to up-to-date information not covered by education material, and

being aware of the particulars and constraints of particular development settings [15], [25].

This alignment is a major step towards the usability of agents in real-life software engineering contexts, as it allows them to exist inside a known development ecosystem (as opposed to existing in a vacuum).

B. EVALUATION METHODOLOGIES AND METRICS

The current overall discussion reveals a significant level of heterogeneity in the methodologies used to evaluate studies reviewed, reflecting the relative infancy of the standardized assessment procedures in this rapidly expanding field and the inherent difficulties of assessing agentic systems that function across various dimensions of software engineering functionality. The most common approach to evaluation is based on benchmarks and is used in 67% of the studies and typically employs established benchmarks like HumanEval, MBPP, and SWE-bench to provide standardized reference points. However, 43% of the inquiries found it essential to design domain-specific evaluation jobs that better represent the unique aspects of their corresponding agentic approaches, and thus indicated that current benchmarks might not be sufficient to cover the entire range of capabilities on their way to maturation. In addition, 29% of the studies included human evaluation aspects to assess such dimensions as code quality, usability, and trust which cannot be entirely quantified by automated metrics, whereas 18% provided results of real-world deployment case studies that provide empirical evidence of actual performance under real working conditions.

The most frequently employed metrics reflect the diverse aspects of agentic system performance that researchers consider important to measure and validate. Pass@k rates for

TABLE 4. Evaluation metrics used in agentic software engineering studies.

Metric Category	Specific Metrics	Usage %	Effectiveness
Code Quality	Pass@k (k=1, 5, 10)	89%	High
	BLEU Score	34%	Medium
	CodeBLEU	28%	Medium
Task Completion	Success Rate	76%	High
	Time to Completion	45%	Medium
	Steps Required	32%	Low
Human Evaluation	Preference Score	63%	High
	Usability Rating	41%	Medium
	Trust Score	29%	Medium
Cost Efficiency	Token Usage	52%	Medium
	API Calls	38%	Medium
	Time Cost	34%	Low

Note: BLEU = Bilingual Evaluation Understudy; CodeBLEU is an adaptation of BLEU for code generation. 'Usage %' reports the percentage of studies in our corpus for which the metric was applicable that employed the metric (e.g., Pass@k among code-generation studies, success rate among autonomous-task studies). The 'Effectiveness' column reflects our qualitative judgement of how informative each metric was reported to be in the primary studies (High = consistently discriminative and aligned with study goals; Medium = partially informative; Low = limited usefulness).

code generation tasks were used by 89% of code generation studies, providing a straightforward measure of functional correctness that can be objectively assessed. Task completion rates for autonomous systems were employed by 76% of multi-agent studies, offering insight into the reliability and effectiveness of complex collaborative processes. Human preference scores for collaboration systems were utilized by 63% of human-agent studies, capturing subjective aspects of system usability and trustworthiness that are crucial for practical adoption. Cost-effectiveness measures were included by 41% of studies, reflecting growing concern about the practical economics of deploying agentic systems in resource-constrained environments.

Table 4 provides a comprehensive overview of evaluation metrics used across different types of agentic software engineering studies.

C. TECHNOLOGICAL ENABLERS

Several key technological advancements have enabled the rapid progress in agentic software engineering:

1) AI PLANNING AND REASONING

More sophisticated reasoning and planning are also essential technological enablers that enable agents to solve complex software engineering problems through strategic decomposition. Chain-of-thought (CoT) prompting—a technique where language models articulate intermediate reasoning steps before generating outputs—provides guided instructions in the form of step-by-step reasoning, and exhibits significant advances in performance in tasks involving logical reasoning, systematic analysis, and sensible problem solving procedures [28], [29], [30]. The techniques enable agents to have a steady flow and consistency in the intricate

development processes comprising numerous interdependent decisions and considerations.

The tree search methods allow the agents to examine various possible solution paths in a systematic manner and predict the outcomes of various methods prior to making a commitment to a particular implementation strategy [21], [31]. This ability can be especially useful in the software engineering environment, where choices made at an early stage of the architecture can have long-term effects on the quality, maintainability, and performance of the system. Planning-guided decoding algorithms assist in converting high-level goals and requirements into specific and actionable steps of implementation and tend to be complementary to formal methods or synthesis methods to ensure soundness and correctness.

These strategies allow agents to solve complex software issues through decomposition of issues into small manageable steps and a systematized thinking on each step.

2) REINFORCEMENT LEARNING FROM FEEDBACK

The systematic feedback and the learning of the results of the execution has been important in enhancing the performance and reliability of the agent over time, which has been a key element in the transformation of the simple task-specific tools to the complex autonomous development systems. Verbal self-reflection enables agents to analyze failures in a methodical way and adjust their strategies to prevent similar mistakes in further scenarios [20]. This ability allows agents to come up with more and more complex problem-solving strategies with experience and thus the building of institutional knowledge that improves the effectiveness of the agents over time.

The use of test-driven reinforcement methods can assist agents in the iterative refinement of their code implementations using systematic analyses of test results, thereby establishing feedback loops that guide agents to more reliable and robust solutions [32], [33], [34]. Execution-guided learning can help agents acquire a better understanding of the execution behavior patterns and debugging problems by observing the real world code execution instead of analyzing the code using only static analysis [35].

These feedback processes play a significant role in boosting the reliability of agent-generated code because closed-loop learning allows agents to learn through making mistakes and successes.

3) NATURAL LANGUAGE INTERFACES

The advanced natural language processing and synthesis functionalities will play a critical role in making human-agent interactions more natural and effective, transforming how software can be developed by both technical and non-technical stakeholders. LMs can help agents interpret the intricate demands in natural language and produce explicit, detailed explanations of how they think and why they make the decisions that they do [5]. This feature closes

the historical divide between human conceptualization and machine execution, making it easier to collaborate in a more natural way.

The development of hybrid communications methods that combine natural language and programming language structures can be used as the means of communication between the agents in a multi-agent system and can provide a complex coordination without losing readability and accuracy [1], [16]. Conversational interfaces are an important step in reducing the barrier to entry of non-technical stakeholders wishing to engage in meaningful communication with software development processes without prior knowledge of programming techniques to a significant level.

The capability to communicate using natural language is a step forward in the world of interfaces as we used to know them, and this allows developing software to be more approachable and allows a better interaction between human beings and agents.

D. APPLICATIONS ACROSS DOMAINS

Agentic software engineering techniques have been applied across various domains and use cases:

1) INTELLIGENT PROGRAMMING ASSISTANTS

Agents based on LLM progressively aid human developers with advanced programming assistants that provide humans with substantial assistance in software development work in various dimensions. Code generation and completion tools improve the productivity of developers by smartly suggesting relevant code snippets, full-fleet functions implementations, and even the entire module structure based on the contextual knowledge about the project needs and code patterns [13], [36], [37]. These tools are not limited to mere template based recommendations but produce code that is functionally valid and contextually appropriate and integrates smoothly with the existing codebases.

The concept of pair programming agents is the next step in the direction of the real-time collaboration of the developers and is engaged to provide clever recommendations, alternative implementation methods, and proactive recognition of possible problems emerging in the process of the development process [19]. Explanation and documentation agents fill the unaddressed but imperative facets of software creation that assist developers to comprehend intricate and already extant code and also produce thorough, precise documentation that stays consistent with the progression of code over time [5].

These assistants have been shown to have great productivity advantages, especially in the case of standard coding and boilerplate code creation.

2) SOFTWARE MAINTENANCE AGENTS

Autonomous agents are now tackling software maintenance jobs that used to demand a lot of human time and expertise. The most ambitious of these are bug-fixing agents. They comb through error logs, look for patterns in failures, pinpoint

the offending lines of code and generate their own fixes so the software keeps running smoothly [22], [38], [39]. Other agents focus on improving code quality. They detect “code smells,” suggest better designs and carry out refactoring work that would otherwise be slow and error-prone for human developers [40]. There are also migration agents that help organisations modernise legacy software by adapting it to new platforms, updated libraries or newer language versions [2]. Recent studies show that large language models can even serve as lightweight testers, reproducing bugs with very little input to aid automated debugging [33]. Taken together, these maintenance agents help keep software up to date while cutting down on the technical debt that can build up in large systems.

3) CO-PROGRAMMING AGENTS

As software projects become more complex, it's now common to see multiple AI agents collaborating on tasks that would be too much for any one person or agent alone. In role-based development, each agent is given a specific job—like a CEO, CTO, lead developer, code reviewer or QA tester—so that the overall team structure mirrors and enhances a traditional human team [11], [16]. Another pattern pairs a “writer” agent with a “critic” agent: one produces the code and the other reviews it, offering suggestions and corrections, which leads to higher-quality results than a single agent working without feedback [26], [41]. Test-driven development agents take this further by generating both the tests and the implementation code in lockstep, ensuring that the requirements, tests and final program stay aligned [18], [32], [42]. In each of these setups, different types of agents bring complementary strengths and build in checks and balances to reduce errors.

4) ADAPTIVE USER INTERFACES

The creation of software through the use of agents to develop or enhance user interfaces is an encouraging move towards enabling people to create software more easily. The natural-language generation agents allow plain language descriptions to be converted into working interfaces so that anyone can create a rich application without having to understand how to write the code [2]. Adaptation agents monitor the interface usage and attend to user feedback and over time adjust the layout and features to suit the needs of users in a better way [17]. The accessibility agents introduce features and compliance improvements, which are automatically added to simplify the usage of software by people with various abilities [6]. The combination of these tools demonstrates how agent-based models can expand the scope of software construction to include more people who are able to compose it and assist interfaces in developing alongside their users.

E. INDUSTRIAL ADOPTION AND PRACTICAL IMPACT

The literature indicates growing, but still limited, industrial experimentation with agentic software engineering. Reported

use cases are often constrained to specific tasks—such as code completion, automated testing, and documentation—rather than fully autonomous end-to-end development workflows. Even in those firms there are major challenges: it is not easy to fit agents into existing workflows and toolchains, and quantitative evidence on the display of the investment is not available yet.

Across 29 of the 61 primary studies, authors reported detailed information on industrial adoption patterns. Within this subset, trust and reliability were the most frequently mentioned barrier (reported in 78% of these studies), followed by integration with legacy systems (64%), lack of standardized evaluation metrics (59%), and skills gaps in development teams (52%). Table 5 summarizes the key drivers, obstacles, and success factors reported in these studies.

The percentages in Table 5 are derived from the 29 studies (47.5% of the 61 primary studies) that reported detailed information on industrial adoption patterns. Each percentage represents the proportion of these 29 studies that mentioned the factor.

F. REPORTED CHALLENGES AND LIMITATIONS

Despite promising advances, the literature identifies several significant challenges and limitations in current agentic software engineering approaches:

1) INTER-AGENT COORDINATION AND CONFLICT RESOLUTION

With the increase in the number of agents in collaborative systems, coordinating various agents brings about a lot of complexity and high possibility of conflicts that may render the systems ineffective. These communication protocols between agents are often informal and not standardized, which is a source of interoperability problems and hinders the possibility of integrating agents created by other teams or organizations [1], [12]. The complexity of a multi-agent system increases its computational costs disproportionately with the complexity of the system since multiple agents take a much larger number of tokens to interact with each other and require more processing time, which may render large-scale implementation uneconomical in practice. In addition, the conflict resolution mechanisms to address the conflict between agents are still at an early stage and most of the existing mechanisms do not include sophisticated methods of addressing conflicting recommendations or unsuitable agent behaviors and actions [1].

These are coordination problems that may constrain the effectiveness of multi-agent systems, especially in cases when team size and complexity of the tasks grow.

2) EVALUATION OF AGENT PERFORMANCE

Evaluation of the effectiveness of agentic software engineering systems is not an easy task and exceeds conventional methods of evaluating software. Conventional measures and quality indicators might not be sufficient to determine

the quality, accuracy, or sustainability of code written by agents. This is particularly so when it comes to complex systems with several agents having various specializations of interest [43]. There is little knowledge about the long-term maintenance of the agent written code. Little is known about the way systems created by agents change with time and what impact they have on maintainability, the growth of technical debt, and long-term quality properties [6]. There is a great difference in consistency and reliability depending on the context. Mastropaolo et al. demonstrated that natural language versions of the same programming task that are semantically equivalent can generate dissimilar code implementations [44]. This brings to the fore basic issues in making behavior of agents reliable and predictable. The lack of standardized evaluation frameworks makes it difficult to compare different approaches and assess progress in the field.

3) SECURITY AND SAFETY IN AUTONOMOUS BEHAVIORS

An autonomous agent raises new security and safety concerns that are beyond the scope of traditional software security. One of the risks is that the agents may develop code with embedded security vulnerabilities that may not be detected during the normal testing processes [45], [46]. Later on, reading or processing sensitive codebases by agents leads to the issue of data privacy. It is particularly so in the environment where proprietary algorithms, business logic or confidential data might be revealed to external AI systems [47]. Furthermore, they can be used maliciously in case agents can be fooled by malicious input or prompt-injection schemes. This manipulation may result in malicious code, backdoor, or other security breaches [17]. These concerns highlight the need for robust security measures and ethical guidelines in the deployment of agentic software engineering systems.

4) HUMAN-AGENT COLLABORATION AND TRUST

The cooperation between human developers and autonomous agents is hindered by many challenges, which have to be addressed to implement them effectively. Among the problems are trust calibration: when developers place the trust in the agents too much, then human supervision is taking a hit; when they place the trust in the agents too little, the automation gains are being wasted [19]. Also, agent decisions tend to be poorly transparent and explainable. This lack of transparency makes it difficult for developers to understand why an agent took a specific action or to effectively intervene when issues arise [39].

Integration into existing development workflows requires substantial adaptation of established processes and tools, creating usability challenges in real-world settings where development teams must balance the benefits of agentic approaches against the costs of workflow disruption [6], [48], [49]. The shift toward agentic development also carries important implications for software engineering education, necessitating comprehensive training programs that equip

TABLE 5. Industrial adoption patterns and barriers.

Factor	Category	Percentage	Impact Level
Adoption Drivers	Productivity Gains	84%	High
	Cost Reduction	67%	Medium
	Developer Satisfaction	58%	Medium
	Competitive Advantage	45%	High
Adoption Barriers	Trust & Reliability	78%	High
	Integration Complexity	64%	High
	Evaluation Standards	59%	Medium
	Skills Gap	52%	Medium
	Security Concerns	48%	High
Success Factors	Gradual Implementation	71%	High
	Training Programs	63%	Medium
	Quality Assurance	57%	High

Note: Percentages are based on the 29 primary studies (47.5% of the corpus) that reported industrial adoption factors; each percentage indicates the proportion of these 29 studies that mentioned the factor.

future developers with the skills needed to collaborate effectively with conversational AI systems and manage AI-assisted development processes [50].

Addressing these human factors is essential for the practical adoption of agentic approaches in real-world software development contexts.

Table 6 provides a structured overview of the major challenges identified in agentic software engineering, categorized by type, severity, and current mitigation strategies.

The ‘Frequency’ column in Table 6 reflects the proportion of the 61 primary studies that reported each challenge category.

V. THREATS TO VALIDITY

We acknowledge several potential threats to the validity of our systematic review:

A. INTERNAL VALIDITY

- **Selection bias:** Despite systematic procedures, the rapidly evolving nature of the field may have led to missed studies, particularly recent preprints or industry reports.
- **Quality assessment subjectivity:** Although we used standardized criteria and multiple reviewers, quality assessment inevitably involves subjective judgments.
- **Data extraction consistency:** While we employed standardized forms and cross-validation, the diversity of study types may have introduced extraction inconsistencies.

B. EXTERNAL VALIDITY

- **Publication bias:** Academic literature may over-represent positive results and novel approaches while under-representing negative results or incremental improvements.
- **Temporal scope:** Our focus on 2022-2025 publications captures the recent surge in agentic approaches but may miss relevant foundational work from earlier periods.

- **Language bias:** Restricting to English publications may have excluded relevant research published in other languages.

C. CONSTRUCT VALIDITY

- **Definition variability:** The term “agentic” is interpreted differently across studies, potentially affecting our categorization and analysis.
- **Evaluation metric diversity:** The lack of standardized evaluation approaches limits our ability to make direct comparisons between studies.
- **Maturity assessment:** Determining the practical maturity of approaches is challenging given the limited long-term deployment data.

D. CONCLUSION VALIDITY

- **Sample size limitations:** With 61 studies, certain subcategories have limited representation, potentially affecting the generalizability of findings within those areas.
- **Heterogeneity effects:** The diversity of approaches, domains, and evaluation methods may limit the validity of cross-study comparisons.

VI. PRACTICAL IMPLICATIONS

According to our analysis, the practical implications of each group of stakeholders are as follows:

A. FOR RESEARCHERS

- **Standardization priority:** Develop standardized evaluation frameworks and benchmarks to enable meaningful comparison across approaches
- **Long-term studies:** Conduct longitudinal studies to assess the maintainability and evolution of agent-generated code
- **Interdisciplinary collaboration:** Engage with HCI, cognitive science, and organizational behavior researchers to address human factors

TABLE 6. Major challenges in agentic software engineering.

Challenge Category	Severity	Frequency	Mitigation Strategy
Agent Coordination	High	73%	Standardized Protocols
Evaluation Standards	High	68%	Benchmark Development
Trust & Explainability	Medium	61%	Transparency Tools
Security & Safety	High	58%	Formal Verification
Human-Agent Collaboration	Medium	54%	Interface Design
Scalability	Medium	47%	Architecture Patterns
Long-term Maintenance	Low	29%	Longitudinal Studies

- **Reproducibility focus:** Ensure research artifacts are openly available and results are reproducible

B. FOR PRACTITIONERS

- **Incremental adoption:** Start with well-established single-agent tools for code completion and testing before moving to complex multi-agent systems
- **Hybrid approaches:** Combine human expertise with agent capabilities rather than pursuing full automation
- **Quality assurance:** Implement rigorous testing and review processes for agent-generated code
- **Skills development:** Invest in training developers to effectively collaborate with AI agents

C. FOR TOOL VENDORS

- **Integration focus:** Prioritize seamless integration with existing development environments and workflows
- **Transparency features:** Develop explainable AI features that help users understand agent decisions
- **Customization support:** Enable adaptation to specific organizational contexts and coding standards
- **Security considerations:** Implement robust security measures to prevent malicious use and protect sensitive code

D. FOR ORGANIZATIONS

- **Strategic planning:** Develop clear strategies for integrating agentic approaches into existing development processes
- **Change management:** Prepare for organizational changes in developer roles and responsibilities
- **Risk assessment:** Conduct thorough risk assessments before deploying autonomous agents in critical systems
- **Performance measurement:** Establish metrics to measure the impact of agentic approaches on productivity and quality

E. PRACTITIONER'S QUICK REFERENCE GUIDE

Table 7 provides actionable guidance for selecting agentic approaches based on task requirements and resource constraints.

VII. FUTURE RESEARCH DIRECTIONS

Based on our analysis of the current state of agentic software engineering and the identified gaps and challenges, as well

as broader research agendas on generative AI in SE [51], we propose several promising directions for future research.

A. STANDARDIZED DEVELOPMENT FRAMEWORKS

A significant gap in the current landscape is the absence of standardized frameworks for developing, deploying, and evaluating agentic software engineering systems, which impedes both research progress and practical adoption. The establishment of unified agent architectures represents a critical need, requiring the development of reference architectures that define standard components, interfaces, and communication protocols specifically designed for software engineering agents [1], [52]. These standardized architectures would provide a common foundation that enables researchers and practitioners to build upon each other's work rather than constantly recreating basic infrastructure components.

The development of comprehensive evaluation benchmarks represents another urgent priority, requiring the creation of systematic assessment frameworks that can evaluate agent performance across diverse software engineering tasks and contexts [43]. These benchmarks must go beyond simple code generation metrics to encompass the full spectrum of software engineering activities including requirements analysis, system design, testing, debugging, and maintenance. Additionally, the creation of interoperability standards would enable seamless collaboration between agents developed by different organizations or using different underlying technologies [23], fostering a more cohesive and collaborative ecosystem for agentic software engineering development.

Standardization would accelerate progress by facilitating direct comparison of approaches, promoting code reuse, and establishing best practices for agent development.

B. REAL-WORLD DEPLOYMENTS AND LONGITUDINAL STUDIES

The majority of investigations on agentic software engineering focus on controlled experiments and benchmark testing. This emphasis creates a huge discrepancy between what is attained by researchers and what is useful in practice. Critical research on agentic techniques in software projects of size and scale is acutely required. This kind of research will be able to provide true insights into their practical effect, constraints, and difficulties in deployment [6]. Large-scale implementations would reveal problems and opportunities

TABLE 7. Practitioner’s quick reference guide.

Task	Tool/Agent	Performance	Cost	Use Case
Code Completion	GitHub Copilot	Pass@1: 65–75%	500–1K tokens	Fast IDE autocomplete
Function Generation	AlphaCode, GPT-4+CoT	Pass@1: 45–55%	2–5K tokens	Algorithmic problems
Repository Tasks	SWE-agent	Success: 35–42%	15–30K tokens	Large codebase navigation
Bug Fixing	RepairAgent	Success: 28–35%	10–20K tokens	Automated debugging
Multi-Step Dev	ChatDev, MetaGPT	Completion: 55–65%	50–100K tokens	End-to-end projects
Code Review	CodeAgent	Detection: 60–70%	3–8K tokens	Automated review
Test Generation	TDD agents	Coverage: 70–85%	5–15K tokens	Unit test creation

Key decision factors: Single-agent tools typically offer faster responses at lower token cost, whereas multi-agent systems often consume several times more tokens but can provide higher-quality solutions on complex tasks [11], [15], [16]. The performance and cost ranges in this table are indicative and synthesized from reported results in representative studies (e.g., [4], [8], [15], [22], [26], [37]) combined with our corpus analysis; they should be interpreted as approximate rather than definitive. In all cases, agent outputs require human review, but the required validation effort will vary by context.

Example adoption path: Phase 1 (Months 1–3): introduce single-agent code completion; Phase 2 (Months 4–6): add specialized tools for testing and documentation; Phase 3 (Months 7–12): pilot multi-agent systems for selected tasks; Phase 4 (Year 2+): integrate mature agentic workflows into CI/CD pipelines. Actual timelines will vary across organizations.

that are not present in laboratory research and small-scale prototypes.

The long-term maintenance performance is an exceptionally significant area of study that is underrepresented, and a systematic investigation of the dynamics of the temporal development of agent-generated code and its implications on the maintainability of the entire system, the accrual of technical debt, and the quality of ongoing quality features is needed [53].

Of equal importance are studies on organizational influence, where the impact of autonomous agents on team dynamics, developer roles, individual job satisfaction, and expansive organizational processes is questioned [6], [47]. These questions should also address underlining questions of the sufficiency of extant software development methodologies, agile, predictive, or reuse-oriented, within the agentic software engineering environment, or whether new methodological paradigms are needed to exploit the potential of agent capabilities in their full.

These studies would provide valuable insights into the real-world applicability and long-term implications of agentic software engineering approaches.

C. INTERDISCIPLINARY INTEGRATION

The agentic software engineering lies on the border of various academic fields, and future developments are likely to manifest through more rigorous and systematic interdisciplinary integration using the knowledge of various research areas. Cognitive science adds considerable knowledge about human problem-solving and cognition, which can add substantially to the agent reasoning capacity by integration of models capturing the mental processes through which expert developers do think and work in practice [28], [30]. The overall knowledge of the cognitive processes may then be used to design agents that complement human thinking patterns and work processes in a better way.

The study of human-computer interaction provides the necessary prerequisites to the creation of more efficient interfaces and cooperative frameworks between humans and autonomous agents, which will overcome the most crucial usability and acceptance issues that currently limit practical implementations. Another critical integration point of software engineering education is the requirement to systematically change the approach to pedagogical practices and curricula to prepare future developers to effectively work with AI-based tools and agents [50]. Additionally, the principles and methodologies existing in software engineering should be periodically updated and extended to control the conduct of agents and to guarantee quality in the AI-influenced development processes [3], [54], [55].

Cross-disciplinary studies can help to enhance agentic practice with diverse views and methods, thus creating stronger and more functional systems [56].

D. ETHICS AND ALIGNMENT IN AGENT BEHAVIOR

The more autonomous and able to make important decisions that impact software quality, security, and functionality, the more ethical operation and alignment with human values agents must become. Achieving effective value alignment strategies is a research priority, and to achieve this efficiently there must be a systematic approach to make sure that the behavior of agents is never out of alignment with human values and organizational preferences in the software development domain [47]. This alignment difficulty also goes beyond mere rule adherence to a more subtle comprehension of context, priorities, and trade-offs of effective software development decision-making.

The development of comprehensive ethical guidelines is another crucial direction of research that would involve creation of substantial principles and practical guidelines of responsible development and deployment of autonomous software engineering agents [17]. Such guidelines should

TABLE 8. Prioritized future research directions.

Research Direction	Urgency	Impact	Feasibility
Standardized Evaluation Frameworks	High	High	Medium
Agent Coordination Protocols	High	High	Medium
Long-term Impact Studies	Medium	High	Low
Human-Agent Interface Design	High	Medium	High
Security & Privacy Frameworks	High	High	Medium
Cross-platform Interoperability	Medium	Medium	High
Educational Curriculum Development	Medium	Medium	High
Ethical Guidelines	High	Medium	Medium

cover such issues as transparency, accountability, fairness and the proper limits of agent autonomy. Further, systematic research on bias mitigation is needed to overcome possible biases in agent behavior caused by any bias in training data, design decisions about the algorithms or the context of organizations in which they are applied [47].

The studies in this field are necessary to create trustful and beneficial agentic systems that can positively impact human agency when developing software.

E. LLM-POWERED AGENTS FOR COLLABORATIVE SOFTWARE ENGINEERING

The potential of LLM-powered collaborative agents is an area of potential interest in future research, with potentially enormous potential to overcome the current single-agent shortcomings. One of the most urgent requirements is the development of advanced coordination mechanisms of agents, as they require advanced coordination frameworks that allow agents to work on complex software tasks and need to collaborate and share an understanding over a protracted period of time [1], [12], [57]. These coordination mechanisms have to deal with dynamic task assignment, conflict solving, and consistency across concurrent streams of development.

The design of cross-functional agent teams is another ambitious line of research, which involves the systematic design of heterogeneous teams of agents (with different skills and knowledge bases) to support end-to-end software development [11], [16], [58]. Moreover, dynamic collaborative models capable of dynamically reconfiguring patterns of collaboration according to varying task demands and the changing capabilities of agents are a complex research problem that would make multi-agent development systems much more flexible and effective [1], [59].

Future work needs to improve the collaboration between the agents with fewer exchanges to cut down on the computational needs. This is in accordance with the recent surveys that detail the broad sceneries and long-term visions of agent systems in software engineering [60].

F. HYBRID AGENT-HUMAN DEVELOPMENT WORKFLOWS

The interaction of the human developers and the autonomous agents is another aspect that should be optimized in the future, and this field is one that needs to be well understood when it comes to the balance of human and agent capacities. The creation of adaptive assistance levels is a major research

issue, and it necessitates the development of agents, which can transition between varying degrees of autonomy with ease, due to the complexity of the tasks, the preferences of the human developers, and other contextual factors. Such systems should be intelligent enough to know when they need to be controlled by human beings and when they should be allowed to operate alone.

The development of integrated development environments that are specifically designed to enable effective human-agent collaboration is another promising path of research [15]. Such environments need to move beyond mere tool integration to offer natural, expressive interfaces that lend themselves to natural patterns of human-agent collaboration. Also, development of effective feedback mechanisms that allow human easy and effective control of agent behavior is a critical research issue [20].

Innovations in this field may result in being especially productive software development teams of human and artificial agents combined with each other, as each of them has their own strengths and may provide a higher productivity and quality than each one separately.

G. AGENT OBSERVABILITY AND EXPLAINABILITY

In order to develop successful collaboration between humans and autonomous agents, it is significant to make decision-making practices of such agents more interpretable and transparent. Explainable agent actions are a fundamental research issue, and complex techniques are needed to allow agents to give concise and full descriptions of how they make decisions and the steps they follow in their reasoning and decision-making processes [39]. Such descriptions should be in the level that is understandable by human developers and at the same time in enough detail to facilitate monitoring and advisory needs.

Another promising area of research is the integration of verified agent outputs, which involves pairing large language models (LLMs) with formal specifications and reactive synthesis techniques to increase the accuracy and dependability of generated code [61]. Creation of powerful debugging features to understand and debug agent failures or unexpected behavior is an important practical requirement. In addition, the creation of good visualizing representations of agent action and decision might lead to an enormous expansion in human opinion potential, e.g., in expert domains such as circuit planning [25].

Explainability would also enhance cooperation as well as making it easier to debug, validate and refine agent systems.

H. REPRESENTATIVE TOOLS AND FRAMEWORKS IN THE LITERATURE

Based on the studies included in our review, we highlight the following tools and frameworks as representative examples that practitioners and researchers have used to instantiate agentic software engineering approaches:

1) FOR CODE GENERATION

- **Single-agent systems:** GitHub Copilot, CodeT5, Alpha-Code for well-defined coding tasks
- **Multi-agent systems:** MetaGPT, ChatDev for complex software development projects

2) FOR SOFTWARE MAINTENANCE

- **Bug fixing:** RepairAgent, SWE-agent for automated program repair
- **Code review:** CodeAgent for automated code review processes

3) FOR RESEARCH AND DEVELOPMENT

- **Evaluation frameworks:** benchmarks such as SWE-bench, HumanEval, and MBPP for standardized assessment (as used in several of the primary studies in our corpus).
- **Development platforms:** popular open-source frameworks (e.g., LangChain, AutoGPT) that are frequently employed to build custom agentic systems in recent work.

We do not provide a comparative evaluation of these tools; rather, we summarize how they are used in existing work as examples of concrete instantiations of the approaches discussed in this review.

Table 8 presents a prioritized list of future research directions based on urgency, impact potential, and feasibility assessments derived from our analysis.

VIII. LIMITATIONS

This systematic review has several limitations that should be considered when interpreting the results. Beyond the threats to validity discussed in the previous section, our review also has broader scope- and evidence-related limitations, which we summarize here.

A. SCOPE LIMITATIONS

- **Temporal scope:** Our focus on 2022-2025 captures recent developments but may miss relevant foundational work from earlier periods
- **Language restriction:** English-only papers may have excluded relevant research in other languages
- **Publication venues:** Academic focus may under-represent industry innovations and proprietary developments

B. METHODOLOGICAL LIMITATIONS

- **Heterogeneity:** The diversity of approaches and evaluation methods limits direct comparisons
- **Quality assessment:** Subjective elements in quality assessment despite standardized criteria
- **Rapid evolution:** The fast-paced nature of the field means some findings may become outdated quickly

C. EVIDENCE LIMITATIONS

- **Limited industrial evidence:** Most studies are academic with limited real-world deployment data
- **Short-term focus:** Lack of long-term studies on maintenance and evolution of agent-generated code
- **Evaluation challenges:** Inconsistent metrics and evaluation approaches across studies
- **Meta-analysis constraints:** While 53 studies reported Pass@k metrics, most presented only aggregate success rates without variance estimates or sample sizes necessary for rigorous meta-analysis. We extracted sufficient data from 28 studies for basic synthesis (Table 4). Publication bias assessment via informal funnel plot inspection suggests studies with negative results may be underrepresented, potentially inflating performance estimates. Future research should adopt standardized reporting (effect sizes with confidence intervals) to enable robust meta-analytic synthesis.

IX. CONCLUSION

This paper has presented a systematic review of the emerging field of agentic software engineering, synthesizing methodologies, techniques, and approaches reported in the literature between 2022 and 2025. Through a stringent selection and analysis procedure, we have compiled the results of 61 strictly selected studies to provide an all-inclusive picture of this rapidly developing field.

A. SUMMARY OF KEY CONTRIBUTIONS

The systematic review provides us with a set of relevant insights that can help to develop knowledge in the field of agentic software engineering, the sphere of knowledge that is rapidly changing and which can become transformational. We have developed a detailed taxonomic scheme that provides a hierarchical identification of agentic software engineering methods, which include goal-oriented agents, multi-agent systems, tool-enhanced agents, and frameworks. This taxonomy serves as an information landmark about the methodological comparison of contrasting approaches, and it thus enables the researcher and practitioners to make wise choices on the applicability of certain approaches in their respective situations and needs.

The development of agentic software engineering has strong trends in our analysis, though, and these consist of a traceable trend toward greater agent autonomy, a growing replacement of single-agent architectures with complex multi-agent systems and an increasingly wide interaction with external tools and development environments. These

tendencies provide useful data on the evolutionary path of the sphere and help to define the areas within which further research and development activities will bring the greatest effect in the future.

We have also tabulated and identified some of the major problems facing the field including the complexity of agent coordination, the necessity to standardize evaluation measures, the issue of security and safety, and the barriers to successful human-agent interaction. This high level of challenge mapping highlights key areas that require special research and development to enable more widespread practical implementation of agentic approaches. In addition, we have outlined the potential future research directions, such as the creation of standardized frameworks, investigations of actual deployments, efforts to integrate across disciplines, and the creation of ethical consideration systems, which is based on the gaps and opportunities of study.

In our analysis, there is robust empirical data of fast field development, 250 percent of publications since 2022 and 2024, and a critical examination of technology uptake trends and success measures in a variety of approaches are provided. We also provide specific, practical researcher, practitioner, tool provider, and organizational suggestions on those who seek to implement or develop agentic strategies in their particular situations.

B. THE TRANSFORMATIVE POTENTIAL OF AGENTIC SOFTWARE ENGINEERING

Our review demonstrates that agentic software engineering has transformative potential for software development practices. The incorporation of autonomous agents, which are driven by large language models, into the software development lifecycle inculcates an increased level of automation, collaboration, and intelligence in the software engineering processes.

Key potential benefits emerging from our analysis include improved developer productivity, as agentic approaches can amplify developer capabilities by automating routine tasks, suggesting candidate solutions, and handling parts of complex implementations that would otherwise require substantial human effort [62]. Improved code quality is another frequently reported benefit, with multi-agent review and testing processes capable of identifying issues and suggesting improvements that might be missed by individual human developers, particularly in complex systems with multiple interdependent components.

Agents may also facilitate more rapid experimentation and innovation by reducing the implementation burden on human developers and allowing them to focus more attention on high-level problem-solving and design. In addition, more natural language interfaces and autonomous agents could make aspects of software development more accessible to non-specialists, potentially broadening participation in software-related activities and enabling new forms of human-computer collaboration in development processes.

C. LONG-TERM IMPLICATIONS

In addition to short-term technological changes, generative artificial intelligence seems to have significant impacts on the long-term software design, deployment, and maintenance. Recent studies of generative AI emphasize the transformative potential of the technology in a variety of fields, and software engineering is one of the most promising areas of application of AI in the future [63]. One of the most important changes is the shift of the roles of developers as they can now deal with routine code assignment, thus allowing human to attend to higher architecture, strategic management and alignment of technical systems with business goals and user demands.

The software methodologies of the past are also bound to change dramatically to support productive agent-human interaction, and novel practices and frameworks will be introduced to achieve the benefit of such hybrid interactions and ensure the full benefits of the combined human and artificial intelligence. Automation will keep growing in scope, and this may involve more and more complicated areas of software engineering that would be in need of much human experience and judgment. With the further development of generative AI, its increasing effect on society and organizations will have a fundamental impact on the way software is developed, administered, and controlled [64].

The software engineering research will be defined by a more intensive interest in the meta-engineering problems, which include designing processes and systems that can guarantee successful collaboration of the agents and humans in the software creation, maintenance and evolution areas. It can be taken as a transition to the research priorities towards optimization not of single development tasks, but the optimization of the whole ecosystem of human-agent collaboration.

D. CONCLUDING REMARKS

Agentic software engineering is a major development in the design, development, and maintenance of software systems, and its effects extend beyond the simple automation of existing processes. The accelerated development as can be seen in the literature that shows a shift of mere aids to complex autonomous multi-agent systems points out to the fact that the transformation is gaining momentum and is likely to go on redefining the software development environment. Despite the significant difficulties that still remain to be addressed, especially in the following aspects coordination protocols, security systems, evaluation systems, and patterns of human-agent collaboration, the potential gains in the form of productivity enhancement, quality improvement, and accessibility expansion are absolutely convincing and warrant continued expenditure of resources on research and development.

The field is currently at a crossroads where standardization efforts, rigorous assessment procedures, and empirical studies are needed to close the gap between research prototypes and practical deployments. The process of converting research prototypes into systems that are production eligible

requires methodical care regarding the basic issues such as ensuring trust calibration, reliability, and human factor integration.

As the field keeps developing, interdisciplinary cooperation between AI researchers, software engineers, cognitive scientists, and HCI specialists will be necessary to achieve the maximum of possibilities of agentic approaches whilst addressing the challenges, associated with them, responsibly. With intelligent development and deployment practices, agentic software engineering can not only improve the existing software practices but also radically redefine the process of software production as a shared activity between humans and progressively more competent artificial agents.

The evidence suggests that organizations are well advised to begin making the necessary preparations to transform themselves by investing in extensive developer education programmes, creating strong evaluation frameworks to evaluate agentic tools and making incremental steps towards adopting agentic practices in the development processes through well managed pilot programmes. It is probable that the future of software engineering will be marked by advanced human-agent interaction, rather than total automation, and thus will require new skills and organisational practices together with new technological systems to ensure that the new paradigm is used responsibly without compromising quality, security, or human agency across the software development lifecycle.

APPENDIX

QUALITY ASSESSMENT CHECKLIST

The following 15-point checklist was used to assess each study. Each criterion was scored 0–3 (0 = not addressed, 1 = partially, 2 = mostly, 3 = fully).

A. CLARITY OF OBJECTIVES (0–3 POINTS)

- 3: Research questions explicitly stated, well-defined, directly relevant
- 2: Research questions stated but lack precision
- 1: Research objectives vague or must be inferred
- 0: No clear research objectives

B. METHODOLOGICAL RIGOR (0–3 POINTS)

- 3: Research design appropriate, well-justified, rigorously executed
- 2: Research design appropriate but lacks justification or detail
- 1: Research design questionable or poorly described
- 0: Methodological approach inadequate

C. VALIDATION APPROACH (0–3 POINTS)

- 3: Comprehensive validation using appropriate benchmarks; results convincing
- 2: Validation present but limited in scope
- 1: Minimal validation; results preliminary
- 0: No validation provided

D. TRANSPARENCY AND REPRODUCIBILITY (0–3 POINTS)

- 3: Sufficient detail for replication; datasets/code publicly available
- 2: Most details provided but gaps in reproducibility
- 1: Limited details; reproducibility questionable
- 0: Insufficient information to reproduce

E. SIGNIFICANCE AND CONTRIBUTION (0–3 POINTS)

- 3: Significant contribution; clear practical relevance and novelty
- 2: Moderate contribution; some novelty or practical value
- 1: Minor contribution; limited novelty
- 0: No clear contribution

Inclusion threshold: $\geq 8/15$ points.

REFERENCES

- [1] J. He, C. Treude, and D. Lo, “LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead,” *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, pp. 1–30, Jun. 2025, doi: [10.1145/3712003](https://doi.org/10.1145/3712003).
- [2] H. Jin, L. Huang, H. Cai, J. Yan, B. Li, and H. Chen, “From LLMs to LLM-based agents for software engineering: A survey of current, challenges and future,” 2024, *arXiv:2408.02479*.
- [3] M. Alenezi and M. Akour, “AI-driven innovations in software engineering: A review of current practices and future directions,” *Appl. Sci.*, vol. 15, no. 3, p. 1344, Jan. 2025. [Online]. Available: <https://www.mdpi.com/2076-3417/15/3/1344>
- [4] Y. Li, “Competition-level code generation with AlphaCode,” *Science*, vol. 378, no. 6624, pp. 1092–1097, Dec. 2022.
- [5] M.-F. Wong, S. Guo, C.-N. Hang, S.-W. Ho, and C.-W. Tan, “Natural language generation and understanding of big code for AI-assisted programming: A review,” *Entropy*, vol. 25, no. 6, p. 888, Jun. 2023.
- [6] J. Sauvola, S. Tarkoma, M. Klemettinen, J. Riekk, and D. Doermann, “Future of software development with generative AI,” *Automated Softw. Eng.*, vol. 31, no. 1, p. 26, May 2024.
- [7] N. Salem, A. Hudaib, K. Al-Tarawneh, H. Salem, A. Tareef, H. Salloum, and M. Mazzara, “A survey on the application of large language models in software engineering,” *Comput. Res. Model.*, vol. 16, no. 7, pp. 1715–1726, Dec. 2024.
- [8] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. M. Zhang, “Large language models for software engineering: Survey and open problems,” in *Proc. IEEE/ACM Int. Conf. Softw. Eng., Future Softw. Eng. (ICSE-FoSE)*, May 2023, pp. 31–53.
- [9] J. Liu, K. Wang, Y. Chen, X. Peng, Z. Chen, L. Zhang, and Y. Lou, “Large language model-based agents for software engineering: A survey,” 2024, *arXiv:2409.02977*.
- [10] A. Yehudai, L. Eden, A. Li, G. Uziel, Y. Zhao, R. Bar-Haim, A. Cohan, and M. Shmueli-Scheuer, “Survey on evaluation of LLM-based agents,” 2025, *arXiv:2503.16416*.
- [11] S. Hong, X. Zheng, J. H. Chen, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, W. Chenglin, and S. Jürgen, “MetaGPT: Meta programming for a multi-agent collaborative framework,” in *Proc. 12th Int. Conf. Learn. Represent.*, 2023, pp. 23247–23275. [Online]. Available: <https://openreview.net/forum?id=VtmBAGCN7o>
- [12] X. Li, S. Wang, S. Zeng, Y. Wu, and Y. Yang, “A survey on LLM-based multi-agent systems: Workflow, infrastructure, and challenges,” *Vicinagearth*, vol. 1, no. 1, p. 9, Oct. 2024.
- [13] R. A. Husein, H. Aburajouh, and C. Catal, “Large language models for code completion: A systematic literature review,” *Comput. Standards Interface*, vol. 92, Mar. 2025, Art. no. 103917. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0920548924000862>
- [14] A. Grishina, V. Liventsev, A. Härmä, and L. Moonen, “Fully autonomous programming using iterative multi-agent debugging with large language models,” *ACM Trans. Evol. Learn. Optim.*, vol. 5, no. 1, pp. 1–37, Mar. 2025, doi: [10.1145/3719351](https://doi.org/10.1145/3719351).

- [15] J. Yang, C. Jimenez-Gomez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024, pp. 50528–50652.
- [16] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative agents for software development," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (Long Papers)*, vol. 1, Bangkok, Thailand, Aug. 2024, pp. 15174–15186. [Online]. Available: <https://aclanthology.org/2024.acl-long.810/>
- [17] M. Esposito, X. Li, S. Moreschini, N. Ahmad, T. Cerny, K. Vaidhyathan, V. Lenarduzzi, and D. Taibi, "Generative AI for software architecture. Applications, trends, challenges, and future directions," 2025, *arXiv:2503.13310v2*.
- [18] B. Chen, F. Zhang, A. Nguyen, D. Zan, Z. Lin, J.-G. Lou, and W. Chen, "CodeT: Code generation with generated tests," 2022, *arXiv:2207.10397*.
- [19] A. Bucaioni, H. Ekedahl, V. Helander, and P. T. Nguyen, "Programming with ChatGPT: How far can we go?" *Mach. Learn. With Appl.*, vol. 15, Mar. 2024, Art. no. 100526. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2666827024000021>
- [20] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," 2023, *arXiv:2303.11366*.
- [21] A. Zhou, K. Yan, M. Shlapentokh-Rothman, H. Wang, and Y.-X. Wang, "Language agent tree search unifies reasoning acting and planning in language models," 2023, *arXiv:2310.04406*.
- [22] I. Bouzenia, P. Devanbu, and M. Pradel, "Repairagent: An autonomous, llm-based agent for program repair," in *Proc. 47th Int. Conf. Softw. Eng. (ICSE)*, 2025, pp. 2188–2200, doi: [10.1109/ICSE55347.2025.00157](https://doi.org/10.1109/ICSE55347.2025.00157).
- [23] Z. Wrona, W. Buchwald, M. Ganzha, M. Paprzycki, F. Leon, N. Noor, and C.-V. Pal, "Overview of software agent platforms available in 2023," *Information*, vol. 14, no. 6, p. 348, Jun. 2023. [Online]. Available: <https://www.mdpi.com/2078-2489/14/6/348>
- [24] Y. Dong, X. Jiang, Z. Jin, and G. Li, "Self-collaboration code generation via ChatGPT," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 7, pp. 1–38, Sep. 2024, doi: [10.1145/3672459](https://doi.org/10.1145/3672459).
- [25] C.-T. Ho, H. Ren, and B. Khailany, "VerilogCoder: Autonomous verilog coding agents with graph-based planning and abstract syntax tree (AST)-based waveform tracing tool," in *Proc. 39th AAAI Conf. Artif. Intell. 37th Conf. Innov. Appl. Artif. Intell. 15th Symp. Educ. Adv. Artif. Intell.*, vol. 39, 2025, pp. 300–307, doi: [10.1609/aaai.v39i1.32007](https://doi.org/10.1609/aaai.v39i1.32007).
- [26] X. Tang, K. Kim, Y. Song, C. Lothritz, B. Li, S. Ezzini, H. Tian, J. Klein, and T. F. Bissey, "CodeAgent: Autonomous communicative agents for code review," 2024, *arXiv:2402.02172*.
- [27] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *Proc. 45th Int. Conf. Softw. Eng.*, May 2023, pp. 1482–1494, doi: [10.1109/ICSE48619.2023.00129](https://doi.org/10.1109/ICSE48619.2023.00129).
- [28] D. Huang, Q. Bu, Y. Qing, and H. Cui, "CodeCoT: Tackling code syntax errors in CoT reasoning for code generation," 2023, *arXiv:2308.08784*.
- [29] G. Yang, Y. Zhou, X. Chen, X. Zhang, T. Y. Zhuo, and T. Chen, "Chain-of-thought in neural code generation: From and for lightweight language models," *IEEE Trans. Softw. Eng.*, vol. 50, no. 9, pp. 2437–2457, Sep. 2024, doi: [10.1109/TSE.2024.3440503](https://doi.org/10.1109/TSE.2024.3440503).
- [30] J. Li, G. Li, Y. Li, and Z. Jin, "Structured chain-of-thought prompting for code generation," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 2, pp. 1–23, Feb. 2025, doi: [10.1145/3690635](https://doi.org/10.1145/3690635).
- [31] S. Zhang, Z. Chen, Y. Shen, M. Ding, J. B. Tenenbaum, and C. Gan, "Planning with large language models for code generation," 2023, *arXiv:2303.05510*.
- [32] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, "AgentCoder: Multi-Agent-based code generation with iterative testing and optimisation," 2023, *arXiv:2312.13010*.
- [33] S. Kang, J. Yoon, and S. Yoo, "Large language models are few-shot testers: Exploring LLM-based general bug reproduction," 2022, *arXiv:2209.11515*.
- [34] A. Ni, S. Iyer, D. Radev, V. Stoyanov, W.-t. Yih, S. I. Wang, and X. V. Lin, "LEVER: Learning to verify language-to-code generation with execution," in *Proc. 40th Int. Conf. Mach. Learn. (ICML) (Proceedings of Machine Learning Research)*, vol. 202, 2023, pp. 26106–26128.
- [35] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," 2023, *arXiv:2304.05128*.
- [36] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen, "RepoCoder: Repository-level code completion through iterative retrieval and generation," 2023, *arXiv:2303.12570*.
- [37] C. E. A. Coello, M. N. Alimam, and R. Kouatly, "Effectiveness of ChatGPT in coding: A comparative analysis of popular large language models," *Digital*, vol. 4, no. 1, pp. 114–125, Jan. 2024. [Online]. Available: <https://www.mdpi.com/2673-6470/4/1/5>
- [38] Z. Fan, X. Gao, M. Mirchev, A. Roychoudhury, and S. H. Tan, "Automated repair of programs from large language models," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, Los Alamitos, CA, USA: IEEE Computer Society, May 2023, pp. 1469–1481, doi: [10.1109/ICSE48619.2023.00128](https://doi.org/10.1109/ICSE48619.2023.00128).
- [39] S. Kang, B. Chen, S. Yoo, and J.-G. Lou, "Explainable automated debugging via large language model-driven scientific debugging," *Empirical Softw. Eng.*, vol. 30, no. 2, p. 128, Mar. 2025, doi: [10.1007/s10664-024-10594-x](https://doi.org/10.1007/s10664-024-10594-x).
- [40] J. Wei, G. Durrett, and I. Dillig, "Coeditor: Leveraging contextual changes for multi-round code auto-editing," 2023, *arXiv:2305.18584*.
- [41] T. Zhang, T. Yu, T. B. Hashimoto, M. Lewis, W.-T. Yih, D. Fried, and S. I. Wang, "Coder reviewer reranking for code generation," 2022, *arXiv:2211.16490*.
- [42] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Trans. Softw. Eng.*, vol. 50, no. 1, pp. 85–105, Jan. 2024, doi: [10.1109/TSE.2023.3334955](https://doi.org/10.1109/TSE.2023.3334955).
- [43] N. Christakis and D. Drikakis, "Evaluating large language models in code generation: INFINITE methodology for defining the inference index," *Appl. Sci.*, vol. 15, no. 7, p. 3784, Mar. 2025. [Online]. Available: <https://www.mdpi.com/2076-3417/15/7/3784>
- [44] A. Mastropaolo, L. Pascarella, E. Guglielmi, M. Ciniselli, S. Scalabrino, R. Oliveto, and G. Bavota, "On the robustness of code generation techniques: An empirical study on GitHub copilot," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, Los Alamitos, CA, USA: IEEE Computer Society, May 2023, pp. 2149–2160, doi: [10.1109/ICSE48619.2023.00181](https://doi.org/10.1109/ICSE48619.2023.00181).
- [45] O. Çetin, E. Ekmekcioglu, B. Arief, and J. Hernandez-Castro, "An empirical evaluation of large language models in static code analysis for PHP vulnerability detection," *JUCS J. Universal Comput. Sci.*, vol. 30, no. 9, pp. 1163–1183, Sep. 2024.
- [46] X. Zhou, S. Cao, X. Sun, and D. Lo, "Large language model for vulnerability detection and repair: Literature review and the road ahead," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, pp. 1–31, Jun. 2025, doi: [10.1145/3708522](https://doi.org/10.1145/3708522).
- [47] M. A. Akbar, A. A. Khan, and P. Liang, "Ethical aspects of ChatGPT in software engineering research," *IEEE Trans. Artif. Intell.*, vol. 6, no. 2, pp. 254–267, Feb. 2025, doi: [10.1109/TAI.2023.3318183](https://doi.org/10.1109/TAI.2023.3318183).
- [48] J. J. Norheim, E. Rebentisch, D. Xiao, L. Draeger, A. Kerbrat, and O. L. de Weck, "Challenges in applying large language models to requirements engineering tasks," *Design Sci.*, vol. 10, p. e16, Jan. 2024.
- [49] D. Hussein, (2024). *Usability of LLMs for Assisting Software Engineering: A Literature Review*. [Online]. Available: <https://elib.dlr.de/210605/>
- [50] C. Sengul, R. Neykova, and G. Destefanis, "Software engineering education in the era of conversational AI: Current trends and future directions," *Frontiers Artif. Intell.*, vol. 7, Aug. 2024, Art. no. 1436350, doi: [10.3389/frai.2024.1436350](https://doi.org/10.3389/frai.2024.1436350).
- [51] A. Nguyen-Duc, B. Cabrero-Daniel, A. Przybylek, C. Arora, D. Khanna, T. Herda, U. Rafiq, J. Melegati, E. Guerra, K.-K. Kemell, M. Saari, Z. Zhang, H. Le, T. Quan, and P. Abrahamsson, "Generative artificial intelligence for software engineering—A research agenda," *Softw., Pract. Exper.*, vol. 55, no. 11, pp. 1806–1843, Nov. 2025, doi: [10.1002/spe.70005](https://doi.org/10.1002/spe.70005).
- [52] Y. Liu, S. K. Lo, Q. Lu, L. Zhu, D. Zhao, X. Xu, S. Harrer, and J. Whittle, "Agent design pattern catalogue: A collection of architectural patterns for foundation model based agents," *J. Syst. Softw.*, vol. 220, Feb. 2025, Art. no. 112278. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0164121224003224>
- [53] F. Zubair, M. Al-Hitmi, and C. Catal, "The use of large language models for program repair," *Comput. Standards Interface*, vol. 93, Apr. 2025, Art. no. 103951. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S092054892400120X>
- [54] A. Hemmat, M. Sharbaf, S. Kolahdouz-Rahimi, K. Lano, and S. Y. Tehrani, "Research directions for using LLM in software requirement engineering: A systematic review," *Frontiers Comput. Sci.*, vol. 7, Mar. 2025, Art. no. 1519437, doi: [10.3389/fcomp.2025.1519437](https://doi.org/10.3389/fcomp.2025.1519437).

- [55] Y. Yang, X. Xia, D. Lo, and J. Grundy, "A survey on deep learning for software engineering," *ACM Comput. Surv.*, vol. 54, no. 10s, pp. 1–73, Sep. 2022, doi: [10.1145/3505243](https://doi.org/10.1145/3505243).
- [56] K. B. Ooi, G. W. H. Tan, M. Al-Emran, M. A. Al-Sharafi, A. Capatina, A. Chakraborty, others, and L. W. Wong, "The potential of generative artificial intelligence across disciplines: Perspectives and future directions," *J. Comput. Inf. Syst.*, vol. 65, no. 1, pp. 76–107, Jan. 2025.
- [57] Z. Rasheed, M. A. Sami, K.-K. Kemell, M. Waseem, M. Saari, K. Systä, and P. Abrahamsson, "CodePori: Large-scale system for autonomous software development using multi-agent technology," 2024, *arXiv:2402.01411*.
- [58] X. Wang et al., "OpenHands: An open platform for AI software developers as generalist agents," 2024, *arXiv:2407.16741*.
- [59] Z. Wu, C. Han, Z. Ding, Z. Weng, Z. Liu, S. Yao, T. Yu, and L. Kong, "OS-copilot: Towards generalist computer agents with self-improvement," 2024, *arXiv:2402.07456*.
- [60] Y. Wang, W. Zhong, Y. Huang, E. Shi, M. Yang, J. Chen, H. Li, Y. Ma, Q. Wang, and Z. Zheng, "Agents in software engineering: Survey, landscape, and vision," *Automated Softw. Eng.*, vol. 32, no. 2, p. 70, Nov. 2025, doi: [10.1007/s10515-025-00544-2](https://doi.org/10.1007/s10515-025-00544-2).
- [61] W. Murphy, N. Holzer, F. Qiao, L. Cui, R. Rothkopf, N. Koenig, and M. Santolucito, "Combining LLM code generation with formal specifications and reactive program synthesis," 2024, *arXiv:2410.19736*.
- [62] C. Ebert and P. Louridas, "Generative AI for software practitioners," *IEEE Softw.*, vol. 40, no. 4, pp. 30–38, Jul. 2023, doi: [10.1109/MS.2023.3265877](https://doi.org/10.1109/MS.2023.3265877).
- [63] L.-Y. Leong, T.-S. Hew, K.-B. Ooi, G. W.-H. Tan, and A. Koohang, "Generative AI: Current status and future directions," *J. Comput. Inf. Syst.*, p. 134, Apr. 2025, doi: [10.1080/08874417.2025.2482571](https://doi.org/10.1080/08874417.2025.2482571).
- [64] V. C. Storey, W. T. Yue, J. L. Zhao, and R. Lukyanenko, "Generative artificial intelligence: Evolving technology, growing societal impact, and opportunities for information systems research," *Inf. Syst. Frontiers*, vol. 27, no. 5, pp. 2081–2102, Oct. 2025, doi: [10.1007/s10796-025-10581-7](https://doi.org/10.1007/s10796-025-10581-7).



interests include the IoT-enabled smart healthcare, secure multimedia systems, and data engineering for data science. A distinguishing aspect of

NESREEN OTOUM received the B.Sc. degree in management information systems from Philadelphia University, Jordan, and the M.Sc. and Ph.D. degrees from Loughborough University, U.K. She has been an Assistant Professor with the Software Engineering Department, Faculty of Information Technology, University of Petra, since 2013. At the University of Petra, she teaches core software engineering courses and supervises undergraduate student projects. Her research

her academic career is her sustained international collaboration. She coordinates multiple international projects funded by Erasmus+ and DAAD, with a focus on sustainability, capacity building, and the development of remote and virtual engineering laboratories. Through these initiatives, she has successfully secured international mobility grants for both staff and students in the Faculty of Information Technology. She is also an Active Member of the Faculty of Information Technology Quality Assurance Committee.



NUHA ELKHALILI received the B.Sc. degree in computer Science from The American University in Cairo, and the M.Sc. and Ph.D. degrees from the School of Computing, University of Leeds, U.K. She is currently a Professor with the Software Engineering Department, Faculty of Information Technology, University of Petra. From 2018 to 2022, she was the Faculty Dean, leading the faculty through the COVID-19 pandemic and the transition to online learning. She also chaired the university's Board of Information Technology Governance, from 2019 to 2020. With more than 22 years of experience teaching software engineering, she has been involved in quality assurance at the university, since 2011, developing procedures for academic program management. She directed the university's e-learning center, from 2018 to 2022, developing guidelines and training staff on e-learning. She has led and participated in numerous internationally and locally funded projects focusing on gender equality, capacity building, and the development of remote virtual engineering laboratories. Her research interests include data engineering for data science, quality assurance for managing academic programs, and e-learning.

...